

UT.10 GESTIÓN DE BASES DE DATOS RELACIONALES

1. INTRODUCCIÓN	2
2. BASES DE DATOS RELACIONALES	2
3. JDBC	3
4. CONECTORES O DRIVERS	3
4.1 INTEGRACIÓN DEL DRIVER PARA ORACLE XE 21c EN UN PROYECTO JAVA UTILIZANDO ECLIPSE	5
5. INSTALACIÓN DE LA BASE DE DATOS, CREACIÓN DE TABLAS Y EL LENGUAJE SQL	8
6. ESTABLECIMIENTO DE CONEXIONES	8
7. EJECUCIÓN DE CONSULTAS SOBRE LA BASE DE DATOS	11
7.1 ADICIÓN DE INFORMACIÓN	12
7.2 RECUPERACIÓN DE INFORMACIÓN	14
7.3 ACTUALIZACIÓN DE INFORMACIÓN	17
7.4 BORRADO DE INFORMACIÓN	18
7.5 GESTIÓN DE TRANSACCIONES (COMMIT Y ROLLBACK)	19
7.6 CIERRE DE CONEXIONES	21
7.7 EXCEPCIONES EN JDBC	23
7.8 EJEMPLO INTEGRADOR	23

1.INTRODUCCIÓN

En un sistema informático es fundamental garantizar la persistencia de la información manejada por un programa, donde **la persistencia consiste en hacer que los datos perduren en el tiempo**. La primera aproximación se consigue mediante el manejo de ficheros. Ahora ha llegado el momento de dar un paso más y aprender a utilizar un sistema más robusto y eficiente.

Cuando la cantidad de datos a gestionar crece y la estructura de estos se complica, el uso de sistemas de ficheros se convierte en una tarea compleja y susceptible de provocar errores. Para este tipo de escenarios se dispone de las bases de datos que ofrecen mejores resultados y son más fáciles de manejar. En este capítulo se someterán a estudio las bases de datos relacionales.

2.BASES DE DATOS RELACIONALES

Una base de datos relacional (BDR) es un sistema organizado de datos que representa la información conforme al modelo relacional y que garantiza la integridad referencial. Está compuesta de un conjunto de tablas en las que se almacena la información de cada una de las entidades y las relaciones existentes entre ellas.

Una **tabla es una serie de filas y columnas**, en la que **cada fila es un registro y cada columna es un campo**. Un **campo** representa un dato de los elementos almacenados en la tabla (nombre del producto, precio, código de barras, etc.). Cada registro, compuesto por tanto por varios campos, representa un elemento de la tabla (un modelo concreto de camiseta, un modelo concreto de pantalón, etc.). En ellas existe el concepto clave primaria, que se corresponde con un atributo o atributos (columna o columnas), que permite identificar de forma unívoca cada tupla o registro de una tabla.

Este tipo de bases de datos son ampliamente utilizadas y existe un gran número de alternativas tanto libres como propietarias. Para este contenido es muy común utilizar una alternativa libre como es MariaDB que es un derivado libre de MySQL. En nuestro caso, debido a que ya tenemos instalada y funcionando una instancia de Oracle XE 21c, aprovecharemos para utilizarla en los ejemplos del capítulo.

Java, mediante JDBC (**Java Database Connectivity**), permite **simplificar el acceso a base de datos relacionales**, proporcionando un único mecanismo mediante el cual las aplicaciones pueden comunicarse con motores de bases de datos. La empresa Sun desarrolló esta API para el acceso a bases de datos, con tres objetivos principales en mente:

- Ser un API con soporte de SQL: poder construir sentencias SQL e insertarlas dentro de llamadas a la API de Java.
- Aprovechar la experiencia de las API de Bases de Datos existentes.
- Ser sencillo.

3. JDBC

[JDBC](#) es una API Java que hace posible ejecutar sentencias SQL en cualquier base de datos relacional. De JDBC podemos decir que:

- Consta de un **conjunto de clases e interfaces** escritas en Java.
- Proporciona un API estándar para desarrollar aplicaciones que usen bases de datos. De esa forma, siempre usarás las mismas clases y métodos para almacenar datos en la base de datos, independientemente de la base de datos.

Con JDBC, no hay que usar mecanismos diferentes para almacenar datos en una base de datos **Access**, o para almacenar datos en una base de datos **Oracle**, etc., sino que **podemos hacer uso del mismo conjunto de técnicas para usar cualquier base de datos relacional**. Gracias a la API JDBC, ejecutar consultas SQL en una base de datos o en otra será algo transparente para el programador.

Cuando se desarrolló JDBC, y debido a la confusión que hubo por la proliferación de API propietarias de acceso a datos, Sun buscó los aspectos de éxito de una API de este tipo, y el ejemplo a seguir fue [ODBC](#) (Open Database Connectivity).

ODBC se desarrolló con la idea de tener un estándar para el acceso a bases de datos en entornos Windows. Aunque la industria ha aceptado ODBC como medio principal para acceso a bases de datos en Windows, ODBC no se adapta bien en el mundo Java, debido a la complejidad que presenta. Factor que junto a otras cosas ha impedido su transición fuera de entornos Windows.

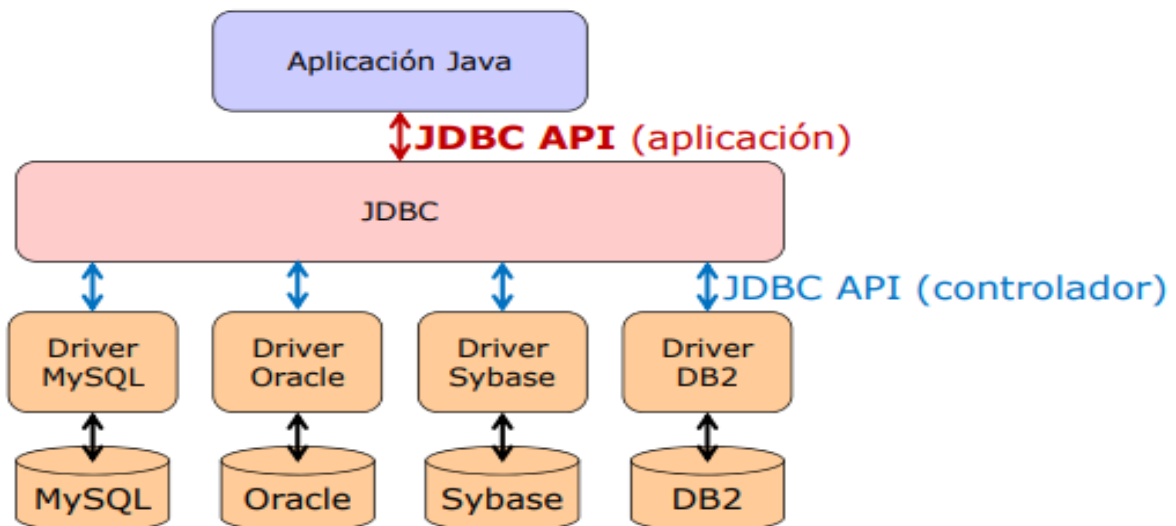
El nivel de abstracción al que trabaja JDBC es alto en comparación con ODBC, la intención de Sun fue que supusiera la base de partida para crear librerías de más alto nivel.

JDBC intenta ser tan simple como sea posible, pero proporcionando a los desarrolladores la máxima flexibilidad.

4. CONECTORES O DRIVERS

La API JDBC viene distribuida en dos paquetes dentro de Java SE:

- `java.sql`, contiene las clases e interfaces principales de la API, y su propósito es facilitar el acceso y procesamiento de datos provenientes de diversas fuentes (generalmente bases de datos relacionales).
- `javax.sql`, contiene clases que complementan a las incluidas en `java.sql`, y aunque está destinado a facilitar el acceso a fuentes de datos en el entorno de servidor, está incluido en Java SE desde la versión 1.4.



Aunque JDBC contiene un buen conjunto de clases, este no es suficiente para poder conectarse y usar una base de datos relacional. Para conectarnos a una base de datos específica, JDBC necesita un **conector o driver** adicional. JDBC es a su vez una especificación que indica cómo debe implementarse dicho conector.

Un conector JDBC es un conjunto de clases que implementan las interfaces de la API JDBC necesarias para conectar a una base de datos específica.

La función del conector es permitir el acceso y la comunicación con el motor de base de datos.

El conector es generalmente implementado por el fabricante del SGBD y consta de una o más librerías java (archivos .jar) que deberemos incorporar a nuestro proyecto.

Cuando se construye una aplicación que accede a una base de datos, **JDBC oculta los detalles específicos de cada base de datos**, de modo que al programar nos debemos preocupar sólo de nuestra aplicación.

Por tanto, aunque el código de nuestra aplicación no depende del driver o conector, puesto que trabajaremos directamente con los paquetes `java.sql` y `javax.sql`, para ejecutar nuestra aplicación y que esta pueda conectarse y usar la base de datos, sí que vamos a necesitar el conector que nos proporcionará el fabricante.

Como se verá más adelante, de cara al desarrollo de nuestra aplicación, JDBC nos ofrece clases e interfaces para:

- Establecer una conexión a una base de datos.
- Ejecutar una consulta.
- Procesar los resultados.

4.1 INTEGRACIÓN DEL DRIVER PARA ORACLE XE 21c EN UN PROYECTO JAVA UTILIZANDO ECLIPSE

La librería de Oracle para conectar nuestras aplicaciones Java con una base de datos Oracle XE 21c es `ojdbc11.jar` (puedes descargarla directamente desde la página <https://www.oracle.com/es/database/technologies/appdev/jdbc-downloads.html>). En nuestro caso utilizaremos el gestor de dependencias Maven para que sea él el encargado de descargarla e integrarla en el proyecto. Para ello puedes seguir los siguientes pasos:

1. Creamos un nuevo proyecto en Eclipse del tipo "Maven project"
2. En la primera pantalla marcamos la opción "Create a simple project (skip archetype selection)". Un "archetype" en Maven es básicamente una plantilla de proyecto. Si dejas la casilla desmarcada, Eclipse intentará descargar un catálogo enorme de plantillas de internet. Al marcar la casilla de proyecto simple, Maven genera un proyecto básico, limpio y sin código de ejemplo innecesario. Creará directamente la estructura de carpetas estándar (`src/main/java` y `src/test/java`) y un archivo `pom.xml` en blanco, que es exactamente lo necesario para empezar a trabajar con JDBC.
3. Una vez que marcada esa casilla y pulsamos "Next". Aparece a una pantalla donde debemos definir la identidad del proyecto. Los campos clave a rellenar son:
 - a. Group Id: Identifica tu proyecto de forma única entre todos los proyectos. Suele seguir la convención de un dominio inverso. Podrías usar algo como `es.instituto.basededatos` o `com.clase.jdbc`.
 - b. Artifact Id: Es el nombre del proyecto y de la carpeta que se creará (por ejemplo, `ConexionOracle` o `EjemploJDBC`).
 - c. Version: Puedes dejar la que viene por defecto (`0.0.1-SNAPSHOT`).
 - d. Packaging: Debe quedarse en `jar`.
4. Al pulsar "Finish" tras rellenar estos datos, Eclipse generará el proyecto. Es en ese momento cuando podrás abrir el archivo `pom.xml` recién creado para pegar la dependencia de `ojdbc11`. Ten en cuenta lo siguiente:
 - a. El driver es simplemente un archivo `.jar` (**`ojdbc11.jar`** para Oracle 21c).
 - b. **No se instala en el sistema operativo**, sino que se añade a las librerías del proyecto (vía Maven/Gradle o añadiéndolo manualmente al *Build Path* en el IDE).
 - c. Si usas Maven, la dependencia es `com.oracle.database.jdbc:ojdbc11`.
 - d. Para utilizar el driver de Oracle 21c, el bloque exacto que tendrán que añadir dentro de la etiqueta `<dependencies>` de su `pom.xml` es este:

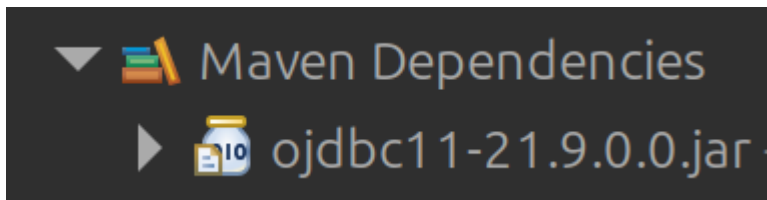
```
<dependencies>
  <dependency>
    <groupId>com.oracle.database.jdbc</groupId>
    <artifactId>ojdbc11</artifactId>
    <version>21.9.0.0</version>
  </dependency>
</dependencies>
```

Si ya tienes la etiqueta dependencies simplemente añade la nueva. Si no, tendrás que crearla dentro de la etiqueta project.

Si tras añadir este elemento al documento XML tienes un error, Eclipse necesita que le des la orden explícita de releer la configuración. Para que aplique los cambios:

1. Haz **clic derecho** sobre la carpeta raíz de tu proyecto en el explorador de paquetes de la izquierda.
2. En el menú desplegable, baja hasta **Maven** y selecciona **Update Project...** (o utiliza el atajo de teclado **Alt + F5**).
3. En la ventana que aparece, asegúrate de que tu proyecto está marcado y pulsa **OK**.

Una vez hecho todo el proceso verás la librería descargada dentro de "Maven Dependencies":



Vamos a ver un pequeño ejemplo de las clases que tendríamos que usar para ejecutar una consulta usando JDBC, échale un vistazo al código, no te preocupes si todavía no lo entiendes todo, más adelante veremos algunas cosas en mayor profundidad:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class PruebaConexion {
    public static void main(String[] args) {

        // 1. Definir las credenciales del esquema que has
creado
        String usuario = "NOMBRE_USUARIO";
        String password = "TU_PASSWORD";

        // 2. Definir las URLs de conexión según la modalidad

        // Modalidad A (Recomendada): Usando Service Name
(XEPDB1)
        // Opcionalmente se puede usar
'//localhost:1521/XEPDB1'
        String urlServiceName =
"jdbc:oracle:thin:@localhost:1521/XEPDB1";
```

```

        // Modalidad B (No recomendada): Usando SID (XE)
        // String urlSID =
        "jdbc:oracle:thin:@localhost:1521:XE";

        // 3. Seleccionar la URL que quieres probar
        String urlSeleccionada = urlServiceName; // Cámbiala a
        urlSID para probar la otra

        System.out.println("Intentando conectar con: " +
        urlSeleccionada);

        // 4. Intentar establecer la conexión
        // Usamos try-with-resources para que la conexión se
        cierre automáticamente
        try (Connection conexion =
        DriverManager.getConnection(urlSeleccionada, usuario,
        password)) {

            if (conexion != null) {
                System.out.println("¡Éxito! Conexión
                establecida correctamente.");
                System.out.println("Esquema actual: " +
                conexion.getSchema());
            }

        } catch (SQLException e) {
            System.err.println("Error al intentar conectar con
            la base de datos.");
            System.err.println("Código de error de Oracle: " +
            e.getErrorCode());
            System.err.println("Mensaje: " + e.getMessage());
        }
    }
}

```

Además de utilizar un gestor de dependencias como Maven o Gradle, también existe la posibilidad de descargar el driver, y añadirlo al classpath del proyecto. El classpath es el conjunto de ubicaciones donde Java puede localizar librerías y software para compilar nuestro proyecto.

5. INSTALACIÓN DE LA BASE DE DATOS, CREACIÓN DE TABLAS Y EL LENGUAJE SQL

Este contenido ya ha sido ampliamente tratado en el módulo de bases de datos, por lo que te remito a los apuntes que tienes disponibles en el aula virtual correspondiente para cualquier duda que te pueda surgir al respecto.

Para nuestros ejemplos crea un nuevo esquema y ejecuta las siguientes instrucciones DDL para crear tres tablas en dicho esquema:

```
CREATE TABLE PRODUCTO (
    ID NUMBER(9) GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    BARCODE VARCHAR2(24 CHAR) NOT NULL,
    NOMBRE VARCHAR2(200 CHAR) NOT NULL,
    PRECIO NUMBER(8,2) NOT NULL
);

CREATE TABLE TICKET (
    ID NUMBER(9) GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    FECHAHORA TIMESTAMP NOT NULL,
    TICKETCERRADO VARCHAR2(1) NOT NULL CHECK (TICKETCERRADO IN ('T',
'F'))
);

CREATE TABLE LINEATICKET (
    ID NUMBER(9) GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    CANTIDAD NUMBER(5) NOT NULL,
    PRECIOVENTA NUMBER(8,2) NOT NULL,
    PRODUCTO_ID NUMBER(9),
    TICKET_ID NUMBER(9),
    FOREIGN KEY(PRODUCTO_ID) REFERENCES PRODUCTO(ID) ON DELETE CASCADE,
    FOREIGN KEY(TICKET_ID) REFERENCES TICKET(ID) ON DELETE CASCADE
);
```

6. ESTABLECIMIENTO DE CONEXIONES

Desde la versión JDBC 4.0 (introducida en Java 6), no es necesario registrar el driver explícitamente. Ten en cuenta que si en alguna ocasión te toca mantener una aplicación muy antigua, sí tendrás que hacer este paso previo utilizando el método `forName` de la clase `Class`. Nosotros omitiremos este paso porque en nuestro entorno moderno no es necesario; actualmente, la clase `DriverManager` descubre y registra automáticamente cualquier driver que hayamos añadido a las dependencias de nuestro proyecto (como hicimos con Maven) y lo utiliza de forma transparente al evaluar la URL de conexión.

Cuando queremos acceder a una base de datos para operar con ella, necesitamos que el conector JDBC esté configurado en nuestro proyecto a dos niveles:

- En un primer nivel es necesario añadir el conector JDBC (la librería .jar) en nuestro proyecto para poder desarrollar la aplicación, sin esa dependencia no podremos probar el código que estamos desarrollando.
- En un segundo nivel, si estamos trabajando con una versión anterior a JDBC 4.0, es necesario registrar el conector JDBC antes de iniciar la conexión.

Una vez que hemos tenido en cuenta los aspectos anteriores, para establecer una conexión con una base de datos debemos utilizar el método `getConnection()` de la clase `DriverManager`. Este método recibe como parámetro la URL de JDBC que identifica a la base de datos con la que queremos realizar la conexión.

Un ejemplo de URL de conexión sería el siguiente:

```
jdbc:oracle:thin:@localhost:1521/XEPDB1
```

En la URL anterior hay varias partes importantes, separadas por dos puntos (:) o por una arroba (@). Veamos cuáles son:

- **jdbc:** indica que se trata de una URL que contiene la información para conectar usando JDBC.
- **oracle:thin:** indica que se trata de una conexión a una base de datos Oracle utilizando su driver "thin" (un controlador ligero escrito enteramente en Java que no requiere software cliente adicional).
- **@localhost:1521/XEPDB1:** indica el servidor, el puerto y el servicio con el que se desea conectar. En este ejemplo, el servidor estaría instalado en la máquina local (localhost), utiliza el puerto por defecto de Oracle (1521) y se conecta a la base de datos conectable (Service Name) llamada XEPDB1.

A diferencia de otros gestores, el driver de Oracle no suele requerir que se añadan parámetros adicionales en la URL (como zonas horarias o codificaciones Unicode) para configuraciones estándar, ya que el driver y la base de datos negocian estos aspectos de forma automática.

Ese es un ejemplo de URL. Si usamos otra base de datos, la URL será diferente. Por ejemplo, si utilizáramos MariaDB sería algo como:

```
jdbc:mariadb://[host][:puerto]/[BD]
```

Esa URL se proporcionará al método `getConnection` para efectuar la conexión, junto con otra información que pueda ser necesaria:

```
String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
String usuario = "prueba";
String password = "prueba";
Connection con = DriverManager.getConnection(url,
usuario, password); // Establecer la conexión
/* Resto del código */
if (con != null) con.close(); // Cerrar la conexión
```

Fíjate que en el ejemplo anterior, aparte de la URL de conexión, se le proporciona el usuario y la contraseña para acceder a la base de datos. En Oracle, como ya sabes, el concepto de usuario coincide con el de esquema, y éste contiene las tablas que utilizarás en tu aplicación (es similar al concepto de base de datos en otros SGBD como MariaDB).

La ejecución de este método devuelve un objeto `Connection` que representa la conexión con la base de datos; **no debes olvidar cerrar la conexión cuando ya no sea necesaria**.

Sin embargo, **el fragmento de código anterior es solo ilustrativo y no compilará por sí solo**. Tienes que tener en cuenta que cuando `DriverManager` itera sobre la colección de drivers para intentar conectar a la URL especificada, pueden ocurrir múltiples fallos (la base de datos está apagada, no hay red, las credenciales son incorrectas, etc.). En Java, estos fallos lanzan una `SQLException`. Al ser una **excepción comprobada** (*checked exception*), **el compilador nos obliga a gestionarla de forma estricta**. Si no lo hacemos, el IDE nos marcará un error y no nos permitirá ejecutar el programa.

Veamos un ejemplo completo, real y funcional de conexión con una base de datos, donde dicha excepción se trata de forma obligatoria y correcta utilizando la estructura try-with-resources para, además, garantizar el cierre automático de la conexión:

```
String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
String usuario = "prueba";
String password = "prueba";
// La conexión se declara e inicializa entre los
paréntesis del try
try (Connection con = DriverManager.getConnection(url,
usuario, password)) {

    /* Consultas a la base de datos. */
    System.out.println("Conexión establecida con
éxito.");

} catch (SQLException ex) {
    System.err.printf("No se pudo conectar a la base de
datos en la URL (%s)\n", url);
    ex.printStackTrace();
}
```

Nota técnica: La estructura *try-with-resources*

Hasta ahora solo hemos utilizado el bloque `try-catch` clásico para gestionar errores. El ejemplo que acabamos de ver utiliza una variante más moderna (introducida en Java 7) llamada ***try-with-resources*** (try con recursos).

Su funcionamiento es muy sencillo y presenta una ventaja enorme a la hora de trabajar con bases de datos o ficheros:

- **Declaración en paréntesis:** En lugar de abrir el bloque `try` directamente con una llave `{`, se añaden unos paréntesis `()`. Dentro de ellos, declaramos e inicializamos los objetos que consumen recursos externos del sistema (como nuestra `Connection`).
- **Cierre automático:** Cualquier objeto instanciado dentro de esos paréntesis que implemente la interfaz `AutoCloseable` de Java (y todas las clases de JDBC lo hacen) **se cerrará de forma automática** en el instante en que termine el bloque `try`. Esto ocurre siempre, independientemente de si el código finalizó con éxito o si saltó al `catch` por un error.

¿Por qué es la opción recomendada? > Si usáramos un `try` tradicional, estaríamos obligados a escribir un bloque `finally` al final de nuestro código, e incluir dentro otro `try-catch` anidado única y exclusivamente para invocar el método `.close()` de la conexión de forma segura. El *try-with-resources* hace todo ese trabajo por nosotros, logrando un código mucho más limpio, fácil de leer y garantizando que jamás dejaremos una conexión abierta por accidente.

7. EJECUCIÓN DE CONSULTAS SOBRE LA BASE DE DATOS

Vamos a aprender a ejecutar distintos tipos de consultas a través de JDBC. Antes de continuar, repasemos lo que ya sabemos hasta ahora. Para operar con una base de datos y ejecutar las consultas necesarias, nuestra aplicación deberá hacer las operaciones siguientes:

- **Integrar el driver correspondiente a tu SGBD** en tu proyecto.
- **Establecer una conexión** con la base de datos.
- **Enviar consultas SQL** y procesar el resultado.
- **Liberar los recursos** al terminar.
- **Gestionar los errores** que se puedan producir.

Bien, una vez repasado lo anterior lo primero que debes saber es que JDBC proporciona tres modelos de sentencias:

- `Statement`: para lanzar consultas sencillas en SQL.
- `PreparedStatement`: para lanzar consultas preparadas, es decir, consultas que tienen parámetros.

- **CallableStatement**: para lanzar la ejecución de procedimientos almacenados en la base de datos.

Además, tienes que tener en cuenta que la API JDBC distingue dos tipos de consultas:

- **Consultas que permiten obtener datos almacenados**: SELECT. Para las consultas tipo SELECT, que obtienen datos de la base de datos, se emplea el método `ResultSet executeQuery(String sql)`. El método anterior retorna un objeto de tipo `ResultSet`, el cual permitirá acceder a los datos resultantes de la ejecución de la consulta para su procesamiento.
- **Consultas que permiten cambiar los datos almacenados**: entre esas consultas tenemos INSERT, UPDATE, DELETE entre otras. Para estas consultas se utiliza el método `executeUpdate(String sql)`. Este método retornará un entero indicando el número de filas afectadas, que serán, dependiendo de la consulta, las filas que se han insertado, actualizado o borrado de la tabla.

Es común, cuando se habla de bases de datos y programación en un mismo contexto, escuchar la expresión "**operaciones CRUD**". Esta expresión equivale en español a "Crear, Leer, Actualizar y Borrar" y hace referencia a las cuatro operaciones básicas que necesita hacer nuestra aplicación para manejar la información:

- **Creación de datos** (consultas tipo INSERT).
- **Lectura de datos** (consultas tipo SELECT).
- **Actualización de datos** (consultas tipo UPDATE).
- **Borrado de datos** (consultas tipo DELETE).

A su vez, otro acrónimo muy común es **DAO**. Este hace referencia a componentes que usamos en nuestro software, como JDBC, para hacer independiente a nuestra aplicación de la base de datos o sistema de almacenamiento de información usado. La idea detrás de este concepto es que un cambio en el almacenamiento afecte lo mínimo posible a nuestra aplicación, minimizando el número de líneas a modificar.

En esa línea, se recomienda enormemente también crear clases específicas para gestionar las operaciones CRUD, de tal forma que dichas clases concentren las operaciones con la base de datos. Tradicionalmente, es común ver que este tipo de clases tienen la terminación DAO para así expresar su propósito, por ejemplo.

`ProductosDAO` podría ser una clase que agrupara las operaciones CRUD con productos.

7.1 ADICIÓN DE INFORMACIÓN

Vamos a aprender cómo añadir registros a una tabla de nuestra base de datos. Por lo que sabemos hasta ahora tendrás que crear una consulta INSERT INTO de SQL, usar un `Statement` o un `PreparedStatement`, y además debemos utilizar el método `executeUpdate()`.

Debido a innumerables motivos, se aconseja usar un PreparedStatement, por lo que vamos a explicar en primer lugar su uso.

La diferencia fundamental entre ambos radica en la seguridad y la limpieza del código. Al usar un Statement tradicional, nos vemos obligados a unir (concatenar) variables de Java directamente dentro de la cadena de texto SQL con comillas y signos '+', lo que hace el código confuso y abre la puerta a graves vulnerabilidades de seguridad conocidas como **Inyección SQL**. Por el contrario, un PreparedStatement precompila la estructura de la consulta en el motor de la base de datos y envía los valores por separado. Al utilizar incógnitas, la base de datos trata esos valores estrictamente como datos (textos, números, fechas) y nunca como código ejecutable, neutralizando cualquier ataque malintencionado.

Como regla general en nuestras aplicaciones, reservaremos el uso de la interfaz **Statement únicamente para consultas estáticas que no requieran parámetros** (es decir, donde no haya que sustituir incógnitas). Ejemplos perfectos para usar un Statement son la lectura de todos los registros de una tabla (SELECT * FROM producto), el borrado masivo de datos (DELETE FROM producto sin cláusula WHERE), o la creación de estructuras (CREATE TABLE). Para cualquier otra consulta que dependa de variables o datos introducidos por el usuario, utilizaremos siempre PreparedStatement por motivos de seguridad.

Un PreparedStatement necesita una consulta SQL que tenga parámetros, los parámetros se indican con "?":

```
String query = "INSERT INTO producto (nombre, barcode, precio)  
VALUES (?, ?, ?)";
```

La consulta anterior tendría 3 parámetros, y cada parámetro sería accesible por la posición que ocupa en la consulta, empezando a numerar el primero en uno (el primer "?" sería el parámetro número 1, el segundo "?" será el parámetro número 2, y así sucesivamente). Después se pueden reemplazar cómodamente con una serie de métodos que proporciona la clase PreparedStatement, veamos algunos de ellos:

- setDouble (int pos, double value) para indicar el valor de un parámetro de tipo double.
- setInt (int pos, int value) para indicar el valor de un parámetro tipo entero.
- setString (int pos, String value) para indicar el valor de un parámetro tipo cadena de texto.
- setDate (int pos, Date value) para indicar el valor de un parámetro tipo fecha (java.sql.Date).
- setTime (int pos, Time value) para indicar el valor de un parámetro tipo hora (java.sql.Time).

Veamos cómo se podría ejecutar entonces la sentencia paramétrica anterior:

```

public static void nuevoProducto(Connection con, String
nombre, String barcode, double precio) {
    String query = "INSERT INTO producto (nombre, barcode,
precio) VALUES (?, ?, ?)";
    if (con != null) {
        try ( PreparedStatement pstmt =
con.prepareStatement(query)) {
            pstmt.setString(1, nombre);
            pstmt.setString(2, barcode);
            pstmt.setDouble(3, precio);
            int registrosAfectados = pstmt.executeUpdate();
            if (registrosAfectados > 0) {
                System.out.println("Producto insertado
correctamente.");
            } else {
                System.out.println("El producto no ha
podido ser insertado.");
            }
        } catch (SQLException ex) {
            System.err.printf("Se ha producido un error al
ejecutar la consulta SQL.");
        }
    }
}

```

En el ejemplo anterior se muestra un método estático que recibe por parámetro la conexión (con) y los datos necesarios para almacenar el producto.

7.2 RECUPERACIÓN DE INFORMACIÓN

Para recuperar la información almacenada en una base de datos tendremos que ejecutar consultas tipo SELECT. Tendremos que preparar una cadena de texto que contenga la consulta SQL, y ejecutar dicha consulta.

Las consultas tipo SELECT se lanzarán principalmente con el método `executeQuery` (aunque también se pueden lanzar con el método `execute`) de la clase `Statement` o `PreparedStatement`. En ambos casos se obtiene un `ResultSet`, que es una clase Java parecida a una lista en la que se aloja el resultado de la consulta. Cada elemento de la lista es uno de los registros de la base de datos que cumple con los requisitos de la consulta.

El `ResultSet` no contiene todos los datos, sino que los va obteniendo de la base de datos según se van pidiendo. La razón de esto es evitar que una consulta que devuelva una cantidad muy elevada de registros, tarde mucho tiempo en obtenerse y sature la memoria del programa.

Con el `ResultSet` hay disponibles una serie de métodos que permiten movernos hacia delante y hacia atrás en las filas, y obtener la información de cada fila. Vamos a ver un ejemplo sencillo en el que se obtiene el id, el nombre y el precio de todos los productos almacenados en la base de datos:

```
public static void mostrarTodosLosProductos(Connection con) {  
    if (con != null) {  
        try ( Statement stmt = con.createStatement()) {  
            ResultSet resultados =  
stmt.executeQuery("SELECT id,nombre,precio FROM producto");  
            while (resultados.next()) {  
                long id = resultados.getLong("id");  
                String nombre =  
resultados.getString("nombre");  
                double precio =  
resultados.getDouble("precio");  
                System.out.printf("%5d %-15s %10.2f\n", id,  
nombre, precio);  
            }  
        } catch (SQLException ex) {  
            System.err.printf("Se ha producido un error al  
ejecutar la consulta SQL.");  
        }  
    }  
}
```

La clase `ResultSet` usa internamente un **cursor** (concepto que ya hemos estudiado en el módulo de bases de datos) que permitirá ir accediendo a cada uno de los registros seleccionados. Inicialmente dicho cursor no apunta a ningún de registro.

En el ejemplo anterior, el método `next()` del `ResultSet` hace que dicho cursor avance al siguiente registro (o que se sitúe en el primer registro si es la primera invocación del método). Si lo consigue, el método `next()` devuelve `true`. Si no lo consigue, porque no haya más registros que leer, entonces devuelve `false`.

Para obtener cada una de las columnas de cada registro, se utilizan los métodos `get`. A dichos métodos habrá que pasar el nombre del campo cuyo valor se desea obtener ("id",

"nombre", "precio" , etc.) o la posición del campo (empezando su numeración en uno). Veamos algunos de los métodos get más importantes:

- `getDouble` para obtener un campo cuyo valor es de tipo `double`.
- `getInt` para obtener un campo cuyo valor es de tipo entero.
- `getString` para obtener un campo cuyo valor es de tipo cadena de texto.
- `getDate` para obtener un campo cuyo valor es de tipo fecha (`java.sql.Date`).
- `getTime` para obtener un campo cuyo valor es de tipo hora (`java.sql.Time`).

En el caso de querer filtrar los registros extraídos por la consulta utilizando una cláusula `WHERE` necesitaremos una consulta paramétrica. Imagina que necesitas obtener todos los productos de la base de datos que tienen un precio mayor a uno pasado por parámetro. Necesitarías una consulta como la siguiente:

```
String query="SELECT id,nombre,precio FROM producto WHERE  
precio >= ?";
```

El procedimiento de ejecución de la consulta anterior necesita hacer uso de las consultas tipo `PreparedStatement`, vistas en apartados anteriores. Veamos un pequeño ejemplo donde se muestran por pantalla los productos cuyo precio es superior a uno dado por parámetro:

```
public static void mostrarProductosPrecioMinimo (Connection  
con, double precioMinimo)  
{  
    String query="SELECT id,nombre,precio FROM producto  
WHERE precio >= ?";  
    if (con!=null) {  
        try ( PreparedStatement pstmt =  
con.prepareStatement(query)) {  
            pstmt.setDouble(1, precioMinimo);  
            if (pstmt.execute()) {  
                ResultSet resultados =  
pstmt.getResultSet();  
                while (resultados.next()) {  
                    long id = resultados.getLong("id");  
                    String nombre =  
resultados.getString("nombre");  
                    double precio =  
resultados.getDouble("precio");  
                    System.out.printf("%5d %-15s %10.2f\n",  
id, nombre, precio);  
                }  
            }  
        } catch (SQLException ex) {
```



```

        System.err.printf("Se ha producido un error al
ejecutar la consulta SQL.");
    }
}
}

```

Igual que en el caso de inserción de registros en la base de datos, aquí se utilizan los métodos set para establecer el valor de los parámetros de la consulta, en este caso, el método setDouble, dado que el precio es de tipo DOUBLE (tiene parte decimal) en la tabla de la base de datos.

Para saber si una instrucción tipo SELECT ha devuelto algún registro no disponemos de un método tipo .isEmpty(). Para saber si el cursor viene vacío, utilizaremos el método **isBeforeFirst()**. Por definición, cuando un ResultSet acaba de nacer, su cursor se sitúa justo *antes* de la primera fila. El método rs.isBeforeFirst() devuelve true si el cursor está en esa posición inicial. Pero la magia está aquí: la documentación oficial de Java especifica que si el ResultSet no contiene ninguna fila, este método devuelve false. Importante: Esta comprobación debe realizarse siempre **antes** de invocar al método next() por primera vez. Si movemos el cursor y luego llamamos a isBeforeFirst(), este devolverá false porque ya no nos encontramos en la posición inicial, lo que podría llevarnos a pensar erróneamente que no hay datos.

```

try (ResultSet rs = pstmt.executeQuery()) {

    // Si no está antes de la primera fila al arrancar, es que
no hay filas
    if (!rs.isBeforeFirst()) {
        System.out.println("No se ha encontrado ningún
producto.");
    } else {
        // Si hay datos, iteramos normalmente
        while (rs.next()) {
            String nombre = rs.getString("nombre");
            // Procesar datos...
        }
    }
}
}

```

7.3 ACTUALIZACIÓN DE INFORMACIÓN

Las consultas de **actualización** son las consultas tipo UPDATE, y tal y como se comentó en apartados anteriores, es necesario ejecutarlas usando el método executeUpdate. Ese método, como ya sabes, retorna el número de registros insertados, actualizados o eliminados, dependiendo del tipo de consulta que se trate.

Imagina la siguiente consulta que cambia el precio de un producto concreto:

```
String query = "UPDATE producto SET precio = ? WHERE id = ?";
```

Ejecutar la consulta anterior es similar a una inserción, veamos como podría ser:

```
public static void actualizarPrecioProducto(Connection con,
long id, double precio) {
    String query = "UPDATE producto SET precio = ? WHERE id
= ?";
    if (con != null) {
        try ( PreparedStatement pstmt =
con.prepareStatement(query)) {
            pstmt.setDouble(1, precio);
            pstmt.setLong(2, id);
            int registrosAfectados = pstmt.executeUpdate();
            if (registrosAfectados > 0)
            {
                System.out.println("El precio del producto
se ha modificado." + precio);
            } else {
                System.out.println("El precio del producto
no se ha modificado, producto no encontrado.");
            }
        } catch (SQLException ex) {
            System.err.printf("Se ha producido un error al
ejecutar la consulta SQL.");
        }
    }
}
```

Fíjate en el uso del valor retornado por el método `executeUpdate`, gracias a dicho valor podemos determinar si la inserción se pudo realizar o no, dado que sabríamos el número de registros que han sido modificados.

7.4 BORRADO DE INFORMACIÓN

Cuando nos interese eliminar registros de una tabla de una base de datos, emplearemos la sentencia SQL `DELETE`. Para eliminar registros de la base de datos recomendamos utilizar `PreparedStatement`, dado que lo más habitual es que haya que pasar algún parámetro a la consulta. Imagina que necesitamos eliminar un producto concreto de la base de datos, deberíamos utilizar una consulta similar a la siguiente:

```
String queryDelete = "DELETE FROM producto WHERE id = ?";
```

En el código anterior se eliminaría un producto con un id interno concreto. En las sentencias de este tipo, normalmente, siempre se usa la clave primaria para especificar qué registro concreto se desea eliminar, y no eliminar otro por error.

Veamos un ejemplo de la ejecución de la sentencia anterior:

```
public static void borrarProducto(Connection con, long id) {  
    String queryDelete = "DELETE FROM producto WHERE id=?";  
    if (con != null) {  
        try (PreparedStatement pstmt =  
con.prepareStatement(queryDelete)) {  
            pstmt.setLong(1, id);  
            int registrosAfectados = pstmt.executeUpdate();  
            if (registrosAfectados > 0) {  
                System.out.println("El producto ha sido  
eliminado correctamente");  
            } else {  
                System.out.println("El producto no ha sido  
eliminado, porque no existe.");  
            }  
        } catch (SQLException ex) {  
            System.err.printf("Se ha producido un error al  
ejecutar la consulta SQL.");  
        }  
    }  
}
```

Fíjate que tal y como ocurría en otros casos anteriores, aprovechamos el valor retornado por el método `executeUpdate` para saber si el registro se eliminó o no.

7.5 GESTIÓN DE TRANSACCIONES (COMMIT Y ROLLBACK)

Por defecto, cuando establecemos una conexión en JDBC, ésta se encuentra en modo *autocommit*. Esto significa que cada vez que ejecutamos una consulta de modificación

(INSERT, UPDATE o DELETE) utilizando el método `executeUpdate()`, los cambios se guardan permanentemente en la base de datos de forma automática e inmediata.

Sin embargo, en aplicaciones reales es muy común que necesitemos ejecutar varias operaciones relacionadas como si fueran un único bloque lógico (una transacción). Si observas la estructura de las tablas creadas al principio del tema, verás que un TICKET está ligado a varias LINEATICKET. Al guardar una venta, debemos insertar la cabecera del ticket y luego sus líneas asociadas. Si la cabecera se guarda pero el programa falla al guardar las líneas, la base de datos quedaría en un estado inconsistente (un ticket sin líneas).

Para garantizar la integridad referencial y la consistencia de los datos, JDBC nos permite desactivar el guardado automático y gestionar la transacción manualmente.

El proceso consta de tres pasos fundamentales utilizando los métodos del objeto `Connection`:

1. **Desactivar el autocommit:** Se indica el inicio de la transacción con `con.setAutoCommit(false);`.
2. **Confirmar (Commit):** Si todas las sentencias SQL (inserciones, actualizaciones o borrados) se ejecutan correctamente, los cambios se hacen definitivos invocando `con.commit();`.
3. **Deshacer (Rollback):** Si se produce alguna excepción durante el proceso, capturada por nuestro bloque `catch`, podemos deshacer todas las operaciones previas de esa transacción llamando a `con.rollback();`.

Veamos un esquema básico de cómo se estructuraría el código en Java:

```
public static void guardarVentaCompleta(Connection con) {
    try {
        // 1. Desactivamos el guardado automático
        con.setAutoCommit(false);
        /* * 2. Ejecutamos varias operaciones (ej. usando
PreparedStatement).
        * Insertamos el TICKET...
        * Insertamos la LINEATICKET 1...
        * Insertamos la LINEATICKET 2...
        */
        // 3. Si llegamos hasta aquí sin errores, confirmamos
        todos los cambios a la vez
        con.commit();
        System.out.println("Venta guardada con éxito en la base
de datos.");
    } catch (SQLException ex) {
```

```

        System.err.println("Se produjo un error durante la
venta. Deshaciendo cambios...");
        try {
            // 4. Si algo falla, deshacemos lo que se hubiera
insertado a medias
            if (con != null) {
                con.rollback();
            }
        } catch (SQLException rollbackEx) {
            System.err.println("Error crítico al intentar hacer
el rollback.");
        }
    } finally {
        try {
            // 5. Opcional pero recomendado: devolver la
conexión a su estado original
            if (con != null) {
                con.setAutoCommit(true);
            }
        } catch (SQLException autoCommitEx) {
            System.err.println("Error al restaurar el
autocommit.");
        }
    }
}
}

```

Es una buena práctica restaurar el **autocommit** a **true** en el bloque **finally** si vamos a seguir utilizando esa misma conexión para otras consultas aisladas dentro del programa.

7.6 CIERRE DE CONEXIONES

Las conexiones a una base de datos consumen muchos recursos en el sistema gestor de bases de datos y también en la misma memoria del sistema donde se ejecuta la aplicación.

Si realizamos un programa que realiza múltiples conexiones a la base de datos, y luego no las cerramos adecuadamente, estaremos ocupando memoria innecesariamente.

Por ello, conviene cerrar las conexiones con el método `close()` siempre que vayan a dejar de ser utilizadas, en lugar de esperar a que el recolector de basura de Java (garbage collector) las elimine.

Veamos un ejemplo de cómo cerrar una conexión a una base de datos de forma adecuada:

```

String url="jdbc:oracle:thin:@localhost:1521/XEPDB1";
String usuario="prueba";

```

```
String password="prueba";
Connection con = null;
try {
    con = DriverManager.getConnection(url,usuario,password);
    /* Consultas a la base de datos. */

} catch (SQLException ex) {
    System.err.printf("No se pudo conectar a la base de datos (%s)\n", url);
    ex.printStackTrace();
} finally {
    if (con!=null) con.close();
}
```

El código anterior puede ser reemplazado por un try-with-resources, tal y como se ha hecho en ejemplos anteriores. Esto tiene una gran ventaja, y es que nos libera de tener que invocar el método `.close()`, dado que esta estructura garantiza el cierre de recursos de forma automática y segura. De hecho, en el ejemplo anterior habría que envolver la llamada al método `close()` dentro de otra estructura try-catch, ya que la llamada puede lanzar a su vez otra excepción `SQLException`.

También es necesario cerrar las sentencias (**Statement** y **PreparedStatement**). De esa forma también liberamos memoria y recursos del sistema. No es un problema en programas que ejecutan pocas consultas, pero en programas que ejecutan miles o millones de consultas se convierte en un problema grave. Como hemos visto en ejemplos anteriores, con esas sentencias también se puede usar un try-with-resources, por lo que vamos a poner un ejemplo alternativo donde no se utiliza dicha estructura:

```
String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
String usuario = "prueba";
String password = "prueba";
Connection con = null;
try {
    con = DriverManager.getConnection(url, usuario, password);

    /* Consultas a la base de datos. */

} catch (SQLException ex) {
    System.err.printf("No se pudo conectar a la base de datos (%s)\n", url);
    ex.printStackTrace();
} finally {
```

```

try {
    if (con != null) {
        con.close();
    }
} catch (SQLException ex) {
    System.err.println("Error al intentar cerrar la
conexión.");
}
}

```

A veces, incluso es conveniente liberar los resultados (ResultSet). No obstante, cuando se cierra la consulta, también se liberan los ResultSet asociados.

7.7 EXCEPCIONES EN JDBC

En todas las aplicaciones en general, y por tanto en las que acceden a bases de datos en particular, nos puede ocurrir con frecuencia que la aplicación no funciona, no muestra los datos de la base de datos que deseábamos, etc.

Es importante capturar las excepciones que puedan ocurrir para que el programa no aborte de manera abrupta. Además, es conveniente tratarlas para que nos den información sobre si el problema es que se está intentando acceder a un servicio que no existe, o que **el motor de la base de datos Oracle (o su Listener) no está arrancado**, o que se ha intentado hacer alguna operación no permitida...

Cuando alguna de las clases que usamos de JDBC se encuentra con un error, generará una excepción tipo SQLException que incluye información muy útil. En especial, cuando se trata de consultas, dado que nos puede dar detalles importantes de cuál es el problema en la consulta.

Es importante que las operaciones de acceso a base de datos estén dentro de un bloque try-catch que gestione las excepciones.

Los dos métodos más útiles de SQLException son el método getMessage(), que permite recoger y mostrar el mensaje de error que ha generado la base de datos (generalmente en inglés), y también el método getSQLState(), que permite obtener un código que identifica el error que se ha producido. También es útil el método getErrorCode(), el cual devuelve un número entero que representa el código de error asociado.

7.8 EJEMPLO INTEGRADOR

Crea una aplicación que gestione la tabla PRODUCTO utilizando una clase que implemente los métodos estáticos vistos en los apartados anteriores. Genera, además, una clase de Utilidades con métodos estáticos que gestione la introducción de datos por teclado, y finalmente escribe una aplicación que, haciendo uso de ambas clases, gestione el CRUD completo.

Clase Utilidades.java

```

import java.util.Scanner;
public class Utilidades {
    private static final Scanner scanner = new
Scanner(System.in);
    public static String leerCadena(String mensaje) {
        System.out.print(mensaje);
        return scanner.nextLine();
    }
    public static double leerDouble(String mensaje) {
        while (true) {
            try {
                System.out.print(mensaje);
                double valor =
Double.parseDouble(scanner.nextLine().replace(",", "."));
                return valor;
            } catch (NumberFormatException e) {
                System.out.println("Error: Introduce un número
decimal válido.");
            }
        }
    }
    public static long leerLong(String mensaje) {
        while (true) {
            try {
                System.out.print(mensaje);
                long valor =
Long.parseLong(scanner.nextLine());
                return valor;
            } catch (NumberFormatException e) {
                System.out.println("Error: Introduce un número
entero válido.");
            }
        }
    }
}

```

Clase CRUD.java

```

import java.sql.Statement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
public class CRUD {

```



```

    public static void nuevoProducto(Connection con, String
nombre, String barcode, double precio) {
        String query = "INSERT INTO producto (nombre, barcode,
precio) VALUES (?, ?, ?)";
        if (con != null) {
            try (PreparedStatement pstmt =
con.prepareStatement(query)) {
                pstmt.setString(1, nombre);
                pstmt.setString(2, barcode);
                pstmt.setDouble(3, precio);
                int registrosAfectados = pstmt.executeUpdate();
                if (registrosAfectados > 0) {
                    System.out.println("Producto insertado
correctamente.");
                } else {
                    System.out.println("El producto no ha
podido ser insertado.");
                }
            } catch (SQLException ex) {
                System.err.println("Se ha producido un error al
ejecutar la consulta SQL de inserción.");
            }
        }
    }

    public static void mostrarTodosLosProductos(Connection con)
{
        if (con != null) {
            try (Statement stmt = con.createStatement();
ResultSet resultados =
stmt.executeQuery("SELECT id, nombre, precio FROM producto"))
            {
                if (!resultados.isBeforeFirst()) {
                    System.out.println("No hay ningún producto
registrado en la base de datos.");
                } else {
                    while (resultados.next()) {
                        long id = resultados.getLong("id");
                        String nombre =
resultados.getString("nombre");
                        double precio =
resultados.getDouble("precio");
                        System.out.printf("%5d %-15s %10.2f%n",
id, nombre, precio);
                    }
                }
            }
        }
    }

```

```

        }
    }
    } catch (SQLException ex) {
        System.err.println("Se ha producido un error al
ejecutar la consulta SQL de lectura.");
    }
}

public static void mostrarProductosPrecioMinimo(Connection
con, double precioMinimo) {
    String query = "SELECT id, nombre, precio FROM producto
WHERE precio >= ?";
    if (con != null) {
        try (PreparedStatement pstmt =
con.prepareStatement(query)) {
            pstmt.setDouble(1, precioMinimo);

            try (ResultSet resultados =
pstmt.executeQuery()) {
                while (resultados.next()) {
                    long id = resultados.getLong("id");
                    String nombre =
resultados.getString("nombre");
                    double precio =
resultados.getDouble("precio");
                    System.out.printf("%5d %-15s %10.2f%n",
id, nombre, precio);
                }
            }
        } catch (SQLException ex) {
            System.err.println("Se ha producido un error al
ejecutar la consulta SQL paramétrica.");
        }
    }
}

public static void actualizarPrecioProducto(Connection con,
long id, double precio) {
    String query = "UPDATE producto SET precio = ? WHERE id
= ?";
    if (con != null) {
        try (PreparedStatement pstmt =
con.prepareStatement(query)) {

```

```

        pstmt.setDouble(1, precio);
        pstmt.setLong(2, id);
        int registrosAfectados = pstmt.executeUpdate();
        if (registrosAfectados > 0) {
            System.out.println("El precio del producto
se ha modificado a: " + precio);
        } else {
            System.out.println("El precio del producto
no se ha modificado. Producto no encontrado.");
        }
    } catch (SQLException ex) {
        System.err.println("Se ha producido un error al
ejecutar la consulta SQL de actualización.");
    }
}

public static void borrarProducto(Connection con, long id)
{
    String queryDelete = "DELETE FROM producto WHERE id =
?";
    if (con != null) {
        try (PreparedStatement pstmt =
con.prepareStatement(queryDelete)) {
            pstmt.setLong(1, id);
            int registrosAfectados = pstmt.executeUpdate();
            if (registrosAfectados > 0) {
                System.out.println("El producto ha sido
eliminado correctamente.");
            } else {
                System.out.println("El producto no ha sido
eliminado porque no existe.");
            }
        } catch (SQLException ex) {
            System.err.println("Se ha producido un error al
ejecutar la consulta SQL de borrado.");
        }
    }
}
}

```

Clase App.java (programa principal):

```
import java.sql.Connection;
```

```

import java.sql.DriverManager;
import java.sql.SQLException;
public class App {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
        String usuario = "TU_USUARIO"; // Sustituir por el
        usuario real
        String password = "TU_PASSWORD"; // Sustituir por la
        contraseña real
        System.out.println("Conectando a la base de datos...");
        try (Connection con = DriverManager.getConnection(url,
        usuario, password)) {
            System.out.println(";Conexión establecida!\n");

            boolean salir = false;

            while (!salir) {
                mostrarMenu();
                long opcion = Utilidades.leerLong("Elige una
opcion: ");

                System.out.println("-----");

                switch ((int) opcion) {
                    case 1:
                        String nombre =
Utilidades.leerCadena("Nombre del producto: ");
                        String barcode =
Utilidades.leerCadena("Código de barras: ");
                        double precio =
Utilidades.leerDouble("Precio: ");
                        CRUD.nuevoProducto(con, nombre,
barcode, precio);
                        break;
                    case 2:
                        System.out.println("--- LISTADO DE
PRODUCTOS ---");
                        CRUD.mostrarTodosLosProductos(con);
                        break;
                    case 3:
                        double precioMin =
Utilidades.leerDouble("Introduce el precio mínimo: ");

```

```

        System.out.println("--- PRODUCTOS
FILTRADOS ---");
        CRUD.mostrarProductosPrecioMinimo(con,
precioMin);
        break;
        case 4:
            long idActualizar =
Utilidades.leerLong("ID del producto a modificar: ");
            double nuevoPrecio =
Utilidades.leerDouble("Nuevo precio: ");
            CRUD.actualizarPrecioProducto(con,
idActualizar, nuevoPrecio);
            break;
        case 5:
            long idBorrar = Utilidades.leerLong("ID
del producto a eliminar: ");
            CRUD.borrarProducto(con, idBorrar);
            break;
        case 0:
            salir = true;
            System.out.println("Saliendo de la
aplicación...");
            break;
        default:
            System.out.println("Opción no válida.
Inténtalo de nuevo.");
    }

    System.out.println("-----\n");
}
} catch (SQLException e) {
    System.err.println("Error fatal de conexión con
Oracle.");
    e.printStackTrace();
}
}

private static void mostrarMenu() {
    System.out.println("==== GESTIÓN DE PRODUCTOS =====");
    System.out.println("1. Añadir nuevo producto");
    System.out.println("2. Mostrar todos los productos");
    System.out.println("3. Filtrar productos por precio
mínimo");
    System.out.println("4. Actualizar precio de un
producto");
}

```

```
        System.out.println("5. Borrar producto");
        System.out.println("0. Salir");
    }
}
```

Ejercicio: Evolucionando nuestra aplicación (Patrón DAO y POJOs)

1. El problema: La separación de responsabilidades

Hasta ahora, hemos construido una clase CRUD completamente funcional. Hace su trabajo: se conecta a la base de datos, ejecuta sentencias SQL y muestra los resultados. Sin embargo, a nivel de arquitectura de software, nuestra clase está cometiendo un "pecado" muy común: **mezcla responsabilidades**.

Actualmente, nuestra clase CRUD tiene instrucciones `System.out.println()` y `System.out.printf()`. Esto significa que está fuertemente atada a la consola. Piensa en el futuro: ¿qué pasaría si el cliente nos pide convertir esta aplicación de consola en una aplicación con ventanas (usando JavaFX) o en una página web? Nuestra clase CRUD actual no nos serviría, porque seguiría imprimiendo los resultados en la consola invisible del servidor en lugar de enviarlos a la nueva interfaz.

Para solucionar esto, vamos a aplicar un principio fundamental en la ingeniería de software: **la separación de responsabilidades** (cada clase debe hacer una sola cosa). La base de datos no debe saber nada de cómo se muestran los datos al usuario, y la interfaz gráfica no debe saber nada de sentencias SQL.

Para lograr esta separación, vamos a utilizar dos conceptos estándar de la industria: los **POJOs** y el patrón **DAO**.

2. Conceptos clave

- **POJO (Plain Old Java Object - Objeto Java Puro y Simple):** Es una clase básica de Java que no depende de ningún framework ni librería externa. Su única función es transportar datos. Para nosotros, representará una fila exacta de nuestra tabla de la base de datos. Solo contendrá atributos privados, constructores, y métodos *getters/setters*.
- **DAO (Data Access Object - Objeto de Acceso a Datos):** Es un patrón de diseño que aísla la lógica de acceso a la base de datos. Nuestra antigua clase CRUD se convertirá en un DAO. A partir de ahora, un DAO **jamás imprimirá nada por pantalla**. Cuando haga una consulta SELECT, extraerá los datos del `ResultSet`, construirá objetos POJO con esos datos, y devolverá una Lista (`ArrayList`) de esos objetos a la clase que lo llamó.

3. Tu misión: Refactorizar el código

Tu objetivo es transformar el proyecto actual aplicando este nuevo diseño. Para ello, debes seguir estos pasos:

Paso 1: Crear la clase del modelo (POJO)

1. Crea una nueva clase llamada `Producto`.
2. Añade como atributos privados los campos que tenemos en la tabla: `id` (long), `nombre` (String), `barcode` (String) y `precio` (double).
3. Genera un constructor vacío y un constructor con todos los parámetros.
4. Genera los métodos `get` y `set` para todos los atributos.
5. (Opcional pero recomendado) Sobrescribe el método `toString()` para darle un formato amigable si imprimimos el objeto directamente.

Paso 2: Transformar el CRUD en un DAO

Renombra tu clase `CRUD` a `ProductoDAO`.

Modifica los nombres y el retorno de los métodos de lectura: Como esta clase ya no va a imprimir nada, cambia los nombres de los métodos a `obtenerTodosLosProductos` y `obtenerProductosPrecioMinimo`. Ya no deben ser `void`, sino que deben devolver una lista (`List<Producto>`).

- *Pista:* Dentro del método, instancia un `ArrayList` vacío. En el bucle `while` (`resultados.next()`), extrae los datos, crea un nuevo objeto `Producto` usando el constructor, y añádelo a la lista. Al final del método, devuelve esa lista. **¡Borra todos los `System.out.println` y `printf` de estos métodos!**

Añade una búsqueda individual: Crea un nuevo método llamado `obtenerProductoPorID` que reciba como parámetro el identificador (long) de un producto. Este método debe ejecutar una consulta `SELECT` filtrando por ese ID y devolver un único objeto de la clase `Producto` (o `null` si la consulta no encuentra ningún registro con ese identificador).

Modifica los métodos de escritura (INSERT, UPDATE, DELETE): Estos métodos tampoco deben imprimir mensajes de "Producto guardado con éxito". En su lugar, haz que devuelvan un valor `boolean` (`true` si el `executeUpdate()` afectó a más de 0 registros, `false` si falló o no encontró el registro).

- *Extra:* Como reto, modifica el método de insertar para que reciba directamente un objeto `Producto` como parámetro en lugar de recibir el nombre, barcode y precio como variables separadas.

Paso 3: Actualizar la interfaz de usuario (Clase App)

1. Ve a tu clase `App` (donde está el `main` y el menú). Ahora esta clase es la única responsable de interactuar con el usuario.
2. Cuando el usuario elija la opción de listar productos, llama al DAO, recoge la lista de productos que te devuelve, y utiliza un bucle `for-each` para recorrer esa lista e imprimir la información de cada producto por pantalla de forma tabulada.

3. Cuando el usuario inserte un producto, haz que la clase App lea los datos por teclado, cree el objeto Producto y se lo pase al DAO para que lo guarde. Luego, evalúa el boolean que devuelve el DAO para imprimir por pantalla si la operación fue un éxito o un fracaso.